Fichas bibliográficas - FORM-FB-01 Fundamentos de Sistemas Distribuidos y Arquitecturas Distribuidas

Steffanny del Rocío Loor Suárez

07 de noviembre de 2026

Ficha 1: Consistencia causal

Resumen

En este artículo los autores abordan el problema de mantener la consistencia causal de los datos entre servidores distribuidos geográficamente, evitando anomalías en las lecturas sin afectar demasiado el rendimiento. Para ello, Freitas et al. [1] realizaron una revisión de la literatura científica, donde examinaron y compararon sistemas que implementan consistencia causal y admiten transacciones de solo lectura. Como resultado, identificaron cuatro estrategias recurrentes: comprobación de dependencias, secuenciadores, protocolos de estabilización y enfoques optimistas. Además, analizaron sus ventajas y limitaciones, destacando que cada estrategia implica compromisos relacionados con la escalabilidad, latencia, coordinación y rendimiento de los sistemas distribuidos.

Dato concreto

Los autores identifican cuatro estrategias recurrentes para implementar consistencia causal: comprobación de dependencias, secuenciadores, protocolos de estabilización y enfoques optimistas.

Ubicación: Página 10, sección 3.5 (Summary), tabla 1: Recurrent Strategies to Implement Causally Consistent Systems [1].

Nota personal

Este artículo me ayudó a comprender que es difícil mantener un equilibrio entre consistencia, disponibilidad y rendimiento en los sistemas distribuidos. Me pareció interesante la consistencia causal de la que se habla en el artículo, así como también las cuatro estrategias identificadas por Freitas et al. [1]: comprobación de dependencias, secuenciadores, protocolos de estabilización y enfoques optimistas, ya que cada una presenta ventajas y limitaciones. En el informe TA-IND-01 utilizaría esta información para analizar cómo un

sistema distribuido mantiene la consistencia de sus datos y qué compromisos existen entre las diferentes alternativas, relacionándolos con los conceptos de CAP y PACELC.

Ficha 2: Tolerancia a fallos y recuperación de nodos

Resumen

Los fallos permanentes de nodos en clústeres de procesamiento pueden ocasionar problemas como interrumpir la ejecución de tareas distribuidas. Para enfrentar esta situación, Reitz y Fohry [2] proponen un mecanismo de puntos de control a nivel de tarea que permite recuperar el trabajo afectado sin reiniciar toda la aplicación. Esta propuesta fue implementada y evaluada mediante cuatro programas de prueba en dos clústeres y los resultados demostraron que sí es posible continuar con la ejecución utilizando los recursos disponibles, aunque mantener esta protección genera una sobrecarga adicional en el tiempo de procesamiento.

Dato concreto

En los experimentos realizados con 1536 trabajadores en el clúster Lichtenberg, Reitz y Fohry registraron una sobrecarga máxima de protección del 43,98 %, mientras que se obtuvo el 28,3 % con 1280 trabajadores en el clúster Goethe. Estos resultados muestran que los programas necesitaron más tiempo para ejecutarse al utilizar el mecanismo de protección mediante puntos de control, en comparación con las ejecuciones sin protección [2].

Ubicación: Página 8 del PDF, apartado *Experiments*, subsección *Failure-Free Runs*, párrafo que comienza con “The results from Fig. 5 are summarized in Fig. 6” [2].

Nota personal

Aprendí un poco más sobre los mecanismos que permiten recuperar las tareas afectadas sin tener que reiniciar toda la aplicación cuando falla un nodo en un sistema distribuido. Utilizar los puntos de control puede aumentar el tiempo de procesamiento, pero ayuda a mantener la continuidad de las tareas ante determinados fallos [2]. En el informe TA-IND-01 podría utilizar esta información para analizar cómo un sistema distribuido puede recuperarse de los fallos y qué consecuencias tienen estos mecanismos en su rendimiento.

Referencias bibliográficas

[1] D. Freitas, P. deGrandis y T. B. Sousa, «A Survey on the State of the Art of Causally Consistent Cloud Systems,» *ACM Computing Surveys*, vol. 57, n.º 9, mayo de 2025, art. núm. 240. DOI: 10.1145/3731444 dirección: https://doi.org/10.1145/3731444

[2] M. Reitz y C. Fohry, «Task-Level Checkpointing and Localized Recovery to Tolerate Permanent Node Failures for Nested Fork–Join Programs in Clusters,» *SN Computer Science*, vol. 5, 2024, art. núm. 320. DOI: 10.1007/s42979-024-02624-8 dirección: https://doi.org/10.1007/s42979-024-02624-8